

CAHIER DE RECETTE — TEST UNITAIRE

U2 — Client UDP de données capteurs

Référence	Fiche U2 — Test unitaire
Composant testé	clientUDP_données.py (port UDP 9091)
Auteur	Noa Orand
Version testée	buffer UDP 8 Mo, défense en profondeur sur trames VIB
Date du test	10 juin 2026

1. Objectif du test

Vérifier que le client UDP reçoit en continu les trames SOUND et VIB émises par l'ESP32, les parse correctement, écrit les fichiers son_N.csv et vib_N.csv au format attendu, et rejette les trames vibration corrompues (nombre de champs incorrect). Mesurer la fréquence d'échantillonnage son effective et le taux de pertes sur une session de 15 secondes.

2. Environnement de test

- Système : Windows 10/11, Python 3.11
- ESP32 flashé avec le firmware corrigé (snprintf, APLL, buffers DMA × 8)
- Wi-Fi : partage de connexion smartphone (PC et ESP32 sur le même réseau)
- Fichiers attendus : data/records/sons/son_N.csv et data/records/vibrations/vib_N.csv
- Préalable : le serveur TCP a été démarré et a indexé une session au préalable (current_session.txt présent)

3. Procédure d'exécution

1. Démarrer le client : `python clientUDP_données.py` (la console affiche le port d'écoute et la taille du buffer UDP).
2. Lancer une capture de 15 secondes depuis l'IHM (commande START vers l'ESP32 via UDP).
3. Pendant la capture, observer la console : taux de trames reçues, alertes éventuelles de rejet VIB.
4. À la fin de la capture (timeout 3 secondes sans trame), le client doit afficher le résumé : nombre de trames son, nombre de trames vib, nombre de trames rejetées, fréquence son effective.
5. Ouvrir son_N.csv : vérifier l'en-tête (Temps, Amplitude) puis le nombre de lignes (~240 000 attendues pour 15 s à 16 kHz).
6. Ouvrir vib_N.csv : vérifier l'en-tête à 14 colonnes (Timestamp + 13 valeurs) et la cohérence des plages de valeurs.
7. Injecter manuellement une trame VIB tronquée à 12 champs pour confirmer le rejet et l'incrément du compteur de trames rejetées.

4. Résultats attendus

- Création des fichiers son_N.csv et vib_N.csv dans les dossiers cibles, conformes au header de référence.
- Fréquence son effective ≥ 15 kHz (cible 16 kHz, tolérance après correction firmware).

- Taux de perte son $\leq 5 \%$ sur 15 secondes en conditions de Wi-Fi stable.
- Trames VIB à 15 champs acceptées ; trames à nombre de champs différent rejetées et comptabilisées.
- Aucun blocage du client en cas de désynchronisation start/stop (timeout 3 s déclenche la fin de capture).

5. Résultat observé

Sur la session de référence (capture n° 47, condition A_vide, niveau 40 %, durée 15 s) : 237 412 lignes dans son_47.csv (fréquence effective 15 827 Hz, soit 98,9 % de la cible 16 kHz, 0,9 % de pertes), 152 lignes dans vib_47.csv (≈ 10 Hz effectif, conforme à la limitation BLE/Wi-Fi). L'injection d'une trame VIB tronquée à 12 champs a été correctement rejetée — le compteur de trames ignorées est passé à 1 et le message « Trame VIB invalide (12 champs au lieu de 15) — rejetée » s'est affiché. Les fichiers générés sont ré-ouvrables avec `numpy.loadtxt` et exploitables par `charger_sessions()`.

6. Verdict

☒ Validé ☐ Non validé

Date : 10 juin 2026 Réalisée par : Noa Orand